[Համալսարանի անվանում]

[Ֆակուլտետ]

Ավարտական աշխատանքի (դիպլոմային նախագծի) միջամբիոնային առաջադրանք

FPGA-ի հիման վրա սենսորային տվյալների հավաքագրման, մշակման և վերլուծության ինտեգրված ապարատածրագրային հարթակի մշակում

FPGA-Based Sensor Data Acquisition, Processing and Analytics Platform

---

Ուսանող 1. [Ուսանողի անուն, ազգանուն]
Միկրոէլեկտրոնիկայի ամբիոն

Ուսանող 2. [Ուսանողի անուն, ազգանուն]
Ծրագրային / Համակարգչային ճարտարագիտության (կամ Ավտոմատացման / Ալգորիթմների) ամբիոն

Ղեկավար 1. [Ղեկավարի անուն, ազգանուն, աստիճան]

Ղեկավար 2. [Ղեկավարի անուն, ազգանուն, աստիճան]

Երևան, 2026

Բովանդակություն

1 Նախագծի համատեքստ **1**

2 Նախագծի ընդհանուր գաղափարը **2**

2.1 Համակարգի ընդհանուր ճարտարապետություն 2

3 Նպատակներ և խնդիրներ **4**

4 FPGA տվյալների գեներացիա և մշակում **5**

5 PS–PL հաղորդակցություն և runtime կոնֆիգուրացիա **7**

6 Տվյալների ֆորմատ և մոդել **9**

7 Առաջարկվող MVP (նվազագույն կենսունակ տարբերակ) **10**

8 Ուսանող 1 — Միկրոէլեկտրոնիկա **12**

9 Ուսանող 2 — Ծրագրային ապահովում / Ալգորիթմներ **14**

10 Ամփոփ ցուցադրման սցենար **15**

11 Ընդլայնումներ (ընտրովի) **16**

Հապավումների ցանկ **18**

Բառարան **19**

Բաժին 1

Նախագծի համատեքստ

Սույն առաջադրանքը սահմանում է **միջամբիոնային բակալավրիատի դիպլոմային նախագիծ**, որը կատարվում է **երկու ուսանողի** կողմից համատեղ.

- **Ուսանող 1 (Միկրոէլեկտրոնիկա)** — թվային նախագծման / FPGA բաղադրիչ:
- **Ուսանող 2 (Ծրագրային / համակարգչային ճարտարագիտություն, Ավտոմատացում, Ալգորիթմներ կամ հարակից ամբիոն)** — ծրագրային ապահովման, տվյալների վերլուծության, վիզուալիզացիայի և մոդելավորման բաղադրիչ:

Ապարատային հենքը՝

- FPGA հարթակ՝ **AMD AUP ZU3**,
- սարքերի ընտանիք՝ **AMD Zynq UltraScale+ MPSoC**,
- նախագծման գործիքներ՝ **AMD Vivado** և **Vitis**:

Նախագիծը պետք է պարունակի էական թվային նախագծման / FPGA բաղադրիչ առաջին ուսանողի համար և միևնույն ժամանակ ապահովի **անկախ** ծրագրային, տվյալների վերլուծության, վիզուալիզացիայի ու մոդելավորման խնդիր երկրորդ ուսանողի համար: Համակարգը պետք է աջակցի **երկկողմանի հաղորդակցություն**.

1. **FPGA → հարթակ.** հավաքագրման և մշակման արդյունքների փոխանցում,
2. **հարթակ → FPGA.** աշխատանքային ռեժիմում (runtime) կոնֆիգուրացիա և կառավարում:

Բաժին 2

Նախագծի ընդհանուր գաղափարը

Նպատակն է մշակել ինտեգրված ապարատա-ծրագրային հարթակ, որտեղ FPGA-ն գեներացնում կամ ստանում է սենսորանման ազդանշաններ, կատարում է իրական ժամանակի նախնական մշակում (preprocessing) և ստացված տվյալները փոխանցում սերվերին: Web հարթակը պահպանում, վիզուալիզացնում, վերլուծում և մոդելավորում է հավաքագրված տվյալները, ինչպես նաև աշխատանքային ռեժիմում կոնֆիգուրացնում է FPGA ապարատը:

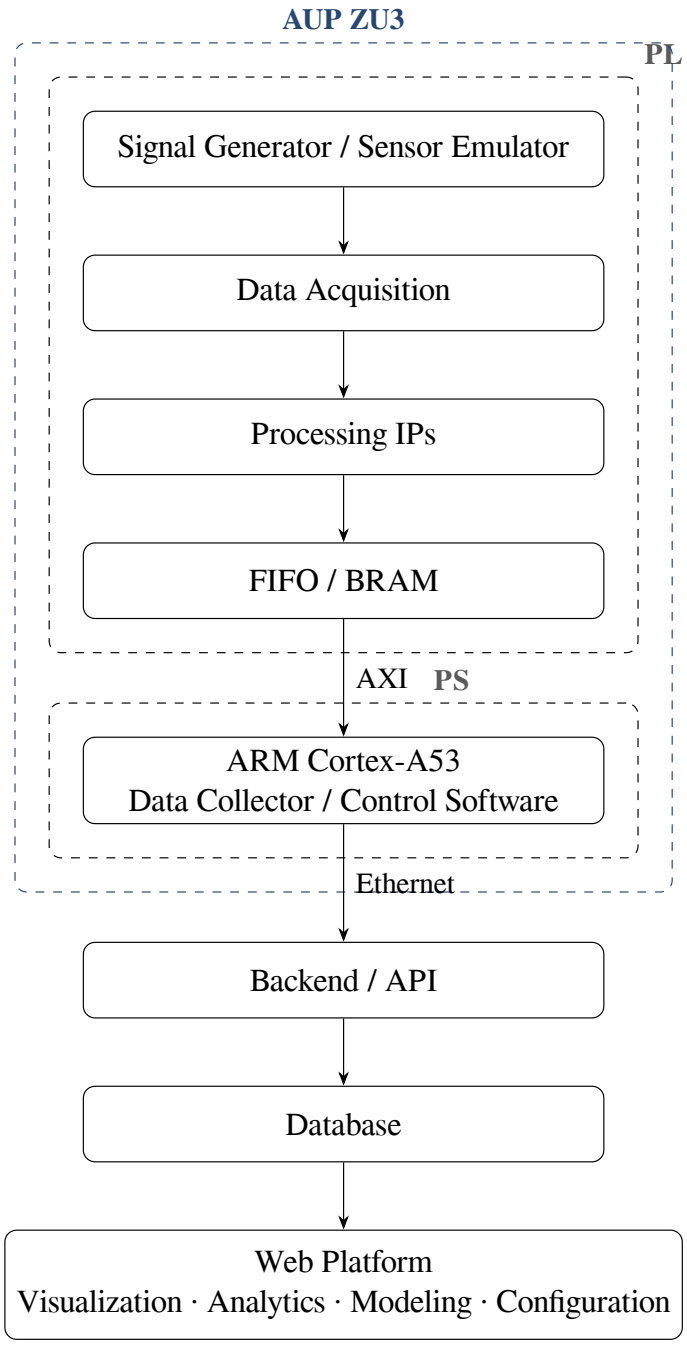
Խնդրի ձևակերպում

Անհրաժեշտ է կառուցել *Web հարթակ* ↔ *Server* ↔ *Zynq PS* ↔ *FPGA* ամբողջական համակարգ, որում FPGA-ն իրական ժամանակում գեներացնում ու մշակում է սենսորային տվյալները, իսկ ծրագրային հարթակը դրանք պահպանում, վերլուծում է և հետադարձ կապով փոխում FPGA-ի պարամետրերը՝ առանց նոր bitstream գեներացնելու:

Սկզբնական իրականացումը **ֆիզիկական սենսորներ չի պահանջում**. սենսորի վարքը կարող է էմուլացվել անմիջապես FPGA-ի ներսում՝ կոնֆիգուրելի RTL ազդանշան-գեներատոր մոդուլներով: Ֆիզիկական սենսորները կարող են միանալ հետագայում՝ որպես ընդլայնում:

2.1 Համակարգի ընդհանուր ճարտարապետություն

Ինչպես ցույց է տրված Նկ. 2.1-ում, FPGA-ի ծրագրավորվող տրամաբանությունը (PL) գեներացնում և մշակում է ազդանշանները, ապա **AXI**-ի միջոցով տվյալները փոխանցվում են մշակող համակարգին (PS՝ ARM Cortex-A53): PS-ը **Ethernet**-ով կապվում է սերվերի հետ, որտեղ գործում են Backend/API-ն, տվյալների բազան և Web հարթակը (վիզուալիզացիա, վերլուծություն, մոդելավորում, կոնֆիգուրացիա):



Նկար 2.1: Համակարգի ընդհանուր ճարտարապետությունը՝ AUP ZU3-ի PL/PS հատվածներից մինչև Web հարթակ.

Բաժին 3

Նպատակներ և խնդիրներ

Նախագծի հիմնական նպատակներն են.

1. FPGA-ի ներսում իրականացնել կոնֆիգուրելի սենսոր-էմուլյացիա (RTL ազդանշան-գեներատորներ):
2. Իրականացնել իրական ժամանակի նախնական մշակում (processing IP-ներ) FPGA-ի վրա:
3. Ապահովել PL→PS տվյալների փոխանցում և PS→սերվեր հաղորդակցություն:
4. Մշակել սերվեր, տվյալների բազա և Web հարթակ՝ պահպանման, վիզուալիզացիայի և վերլուծության համար:
5. Ապահովել FPGA-ի **runtime կոնֆիգուրացիա** Web հարթակից՝ AXI4-Lite ռեգիստրների միջոցով:
6. Ցուցադրել համակարգի ամբողջական աշխատանքը՝ փորձի (experiment) կազմակերպման, տվյալների հավաքագրման և վերլուծության միջոցով:

Բաժին 4

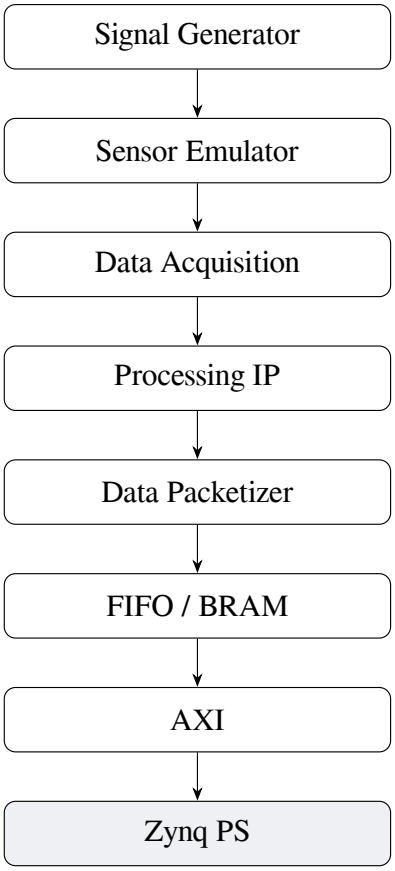
FPGA տվյալների գեներացիա և մշակում

Սկզբնական դիպլոմային տարբերակում իրական սենսորները պարտադիր չեն: Միկրոէլեկտրոնիկայի ուսանողը պետք է իրականացնի **կոնֆիգուրելի ազդանշան-գեներատոր / սենսոր-էմուլյատոր** RTL մոդուլներ, որոնք FPGA-ի ներսում մոդելավորում են սենսորի իրատեսական վարքը: Հնարավոր սենսորների տիպերը՝ ջերմաստիճան (Temperature), ճնշում (Pressure), վիբրացիա (Vibration), ընդհանրական անալոգային սենսոր և այլ կոնֆիգուրելի մոդելներ:

Յուրաքանչյուր սենսորի համար առանձին ապարատ իրականացնելու փոխարեն նախատեսվում են **վերաօգտագործելի ազդանշան-գեներացման պրիմիտիվներ**.

- **Ramp Generator** — աստիճանաբար աճող/նվազող արժեքներ (ջերմաստիճանի, ճնշման, դանդաղ փոփոխվող մեծությունների համար):
- **Periodic Signal Generator** — պարբերական ազդանշաններ (վիբրացիա և այլ կրկնվող երևույթներ):
- **Pseudo-Random / Noise Generator** — LFSR-ի հիման վրա, կոնֆիգուրելի աղմուկ ավելացնելու համար:
- **Anomaly / Fault Generator** — ոչ նորմալ վարքի ներարկում (օր.՝ գերտաքացման կամ բարձր վիբրացիայի իրադարձություն), ինչը տվյալները դարձնում է օգտակար անոմալիաների հայտնաբերման փորձերի համար:

Signal Generator-ի կոնֆիգուրելի պարամետրերն են՝ ազդանշանի տիպ, ամպլիտուդ, հաճախություն, բազային արժեք, նմուշառման արագություն (sample rate), աղմուկի մակարդակ, անոմալիայի միացում և մեծություն: **Sensor Emulator**-ը ընդհանրական ազդանշանը վերածում



Նկար 4.1: FPGA-ի RTL մշակման հնարավոր փոխակրիչը (pipeline).

է սենսորանման արժեքների՝ scaling, offset, saturation, աղմուկի ու անոմալիայի ներարկման միջոցով:

FPGA-ն պետք է կատարի հիմնական իրական ժամանակի մշակում (**Processing IP**-ներ). հնարավոր մոդուլներ՝ Moving Average, Min/Max Detector, Threshold Detector, Event Detector, FIR Filter, Statistical accumulator: Բակալավրիատի ծավալի համար ողջամիտ MVP-ն պարունակում է մոտավորապես **3 մշակող մոդուլ**:

Բաժին 5

PS–PL հաղորդակցություն և runtime կոնֆիգուրացիա

Zynq-ի մշակող համակարգը (PS) ծրագրավորվող տրամաբանության (PL) հետ կապվում է **AXI**-ի միջոցով՝ երկու ուղիներով.

- **Data Path** (AXI-Stream / FIFO) — գեներացված/մշակված տվյալների փոխանցում PL→PS:
- **Control Path** (AXI4-Lite) — FPGA մոդուլների կոնֆիգուրացիա՝ ռեգիստրների միջոցով:

Կարևոր պահանջ է, որ Web հարթակը կարողանա փոխել FPGA-ի վարքը **առանց bitstream վերագեներացնելու՝** AXI4-Lite կոնֆիգուրացիոն ռեգիստրների միջոցով: Ռեգիստրների քարտեզի (register map) օրինակ.

Աղյուսակ 5.1: AXI4-Lite կոնֆիգուրացիոն ռեգիստրների քարտեզի օրինակ.

Հասցե	Ռեգիստր	Հասցե	Ռեգիստր
0x00	CONTROL	0x18	NOISE_LEVEL
0x04	SENSOR_TYPE	0x1C	THRESHOLD
0x08	SAMPLE_RATE	0x20	FILTER_ENABLE
0x0C	SIGNAL_TYPE	0x24	FILTER_WINDOW
0x10	AMPLITUDE	0x28	ANOMALY_ENABLE
0x14	FREQUENCY	0x2C	ANOMALY_LEVEL

Կոնֆիգուրացիայի ուղին է՝ Web Platform → Backend/API → Ethernet → Zynq PS → AXI4-Lite → FPGA ռեգիստրներ → RTL մոդուլներ: Սա *runtime* կոնֆիգուրացիա է և Vivado սինթեզ կամ նոր bitstream չի պահանջում:

Նշում

Պետք է հստակ տարբերել **runtime կոնֆիգուրացիան** (արդեն իրականացված ապարատի պարամետրերի փոփոխություն՝ պարտադիր) և **FPGA reconfiguration**-ը (նախապես գեներացված bitstream-ների միջև ընտրություն՝ ընդլայնում, ոչ MVP):

Բուֆերացումն իրականացվում է FIFO/BRAM-ով. սկզբնական տարբերակում՝ Processing IP → AXI FIFO → Zynq PS: Բարձր տվյալահոսքի դեպքում որպես ընդլայնում հնարավոր է AXI4-Stream → AXI DMA → DDR ուղին (DMA-ն դիտվում է որպես ընդլայնում, ոչ պարտադիր MVP պահանջ):

Բաժին 6

Տվյալների ֆորմատ և մոդել

FPGA/PS տվյալները սերվեր ուղարկելուց առաջ ձևափոխվում են կառուցվածքային ֆորմատի: Օրինակ՝

```
{
  "device": "AUP_ZU3_01",
  "experiment_id": 12,
  "sensor": "vibration",
  "timestamp": 1789063200,
  "sequence": 1524,
  "raw_value": 728,
  "processed_value": 681,
  "status": "normal"
}
```

PS-ը այս տվյալները կարող է ուղարկել API-ին, օրինակ՝ `POST /api/v1/data`:

Տվյալների բազան պետք է աջակցի **փորձերի (experiments)** հասկացությունը, ոչ թե պահի միայն առանձին արժեքներ: Հայեցակարգային մոդելը.

- **Experiment** — `experiment_id`, `device_id`, `configuration`, `start_time`, `end_time`:
- **Sensor** — `sensor_id`, `sensor_type`, `sample_rate`, `unit`:
- **Sample** — `experiment_id`, `sensor_id`, `timestamp`, `sequence_number`, `raw_value`, `processed_value`, `status`:

Այս մոդելը թույլ է տալիս հետագայում համեմատել փորձերը (օր.՝ Experiment #12 vs #18):

Բաժին 7

Առաջարկվող MVP (նվազագույն կենսունակ տարբերակ)

Բակալավրիատի ծավալը իրատեսական պահելու համար առաջին տարբերակը պետք է պարունակի.

Ապարատային մաս.

- մեկ AUP ZU3 հարթակ,
- 3 էմուլացված սենսորի տիպ՝ Temperature, Pressure, Vibration,
- վերաօգտագործելի գեներատորներ՝ Ramp, Periodic, LFSR/Noise, Anomaly injection,
- FIFO/BRAM բուֆերացում,
- մոտ 3 processing IP՝ Moving Average, Min/Max, Threshold/Event Detection,
- AXI4-Lite կոնֆիգուրացիա,
- PL→PS տվյալների փոխանցում,
- Ethernet հաղորդակցություն:

Ծրագրային մաս.

- մեկ սերվերային համակարգիչ (Backend/API, տվյալների բազա),
- Web dashboard՝ փորձի կոնֆիգուրացիայով,
- տվյալների վիզուալիզացիա (live և պատմական),
- փորձերի համեմատում,

- հիմնական վիճակագրական վերլուծություն,
- մեկ մոդելավորման / անոմալիաների հայտնաբերման կիրառություն:

Բաժին 8

Ուսանող 1 — Միկրոէլեկտրոնիկա

Անհատական թեմա.
AUP ZU3 FPGA-ի հիման վրա սենսորային տվյալների գեներացման, հավաքագրման և նախնական մշակման ապարատային համակարգի նախագծում.

Հիմնական պարտականությունները.

1. Կոնֆիգուրելի RTL ազդանշան-գեներատորների նախագծում:
2. Սենսոր-էմուլյացիայի տրամաբանության իրականացում:
3. LFSR/աղմուկի գեներացիայի իրականացում:
4. Անոմալիաների/անսարքությունների ներարկման իրականացում:
5. Տվյալների հավաքագրման (data acquisition) տրամաբանության մշակում:
6. FIFO/BRAM բուֆերացման իրականացում:
7. Կոնֆիգուրելի processing IP-ների մշակում:
8. AXI-ի հիման վրա տվյալների հաղորդակցության իրականացում:
9. AXI4-Lite կոնֆիգուրացիոն ռեգիստրների իրականացում:
10. PL-ի ինտեգրում Zynq PS-ի հետ:
11. Testbench-երի մշակում և RTL-ի վերիֆիկացիա:
12. Vivado սինթեզ, implementation և timing analysis:
13. LUT/FF/DSP/BRAM օգտագործման վերլուծություն:
14. Bitstream-ի գեներացում և ամբողջական նախագծի վալիդացիա AUP ZU3-ի վրա:

15. FPGA/PS տվյալների ինտերֆեյսի և կառուցվածքային ելքային ֆորմատի սահմանում:

Առաջին ուսանողի դիպլոմը պետք է հիմնականում կենտրոնանա թվային նախագծման, RTL-ի, FPGA ճարտարապետության, data/control ուղիների, հիշողության/բուֆերացման, AXI-ի, վերիֆիկացիայի, սինթեզի, implementation-ի, timing-ի և ապարատային վալիդացիայի վրա:

Բաժին 9

Ուսանող 2 — Ծրագրային ապահովում / Ալգորիթմներ

Անհատական թեմա.
FPGA-ից ստացվող սենսորային տվյալների պահպանման, վիզուալիզացիայի և վերլուծության ծրագրային հարթակի մշակում.

Հիմնական պարտականությունները.

1. Տվյալների բազայի ճարտարապետության նախագծում:
2. Backend/API-ի մշակում և FPGA/PS տվյալների ընդունում:
3. Փորձերի և սենսորային նմուշների պահպանում:
4. Web հավելվածի մշակում՝ փորձի կոնֆիգուրացիայի ինտերֆեյսով:
5. Կոնֆիգուրացիոն հրամանների ուղարկում AUP ZU3-ին:
6. Live և պատմական տվյալների վիզուալիզացիա, փորձերի համեմատում:
7. Վիճակագրական վերլուծության իրականացում (mean, variance, std, և այլն):
8. Վերլուծության/մոդելավորման բաղադրիչի իրականացում:
9. (Ընտրովի) անոմալիաների հայտնաբերում կամ կանխատեսում (prediction):
10. (Ընտրովի) վերլուծության արդյունքներով FPGA-ի պարամետրերի ավտոմատ փոփոխում:

Երկրորդ ուսանողի աշխատանքը պետք է մնա բավականաչափ անկախ RTL-ի մշակումից. հիմնական շեշտը՝ տվյալների բազա, Backend, Web հարթակ, վիզուալիզացիա, վիճակագրություն և վերլուծություն/մոդելավորում: Հետաքրքիր համեմատություն է առաջանում FPGA-ի իրական ժամանակի պարզ հայտնաբերման (threshold) և սերվերային ավելի բարդ, պատմական տվյալների վրա հիմնված վերլուծության միջև:

Բաժին 10

Ամփոփ ցուցադրման սցենար

Դիպլոմի վերջնական ցուցադրման հնարավոր սցենար.

1. Օգտվողը բացում է Web հարթակը և ընտրում **Vibration** սենսորը և **Periodic** ազդանշանը:
2. Կոնֆիգուրացնում է պարամետրերը՝ sample rate, ամպլիտուդ, աղմուկ, threshold, filter, anomaly, և սեղմում **Start Experiment**:
3. Backend-ը կոնֆիգուրացիան ուղարկում է AUP ZU3-ին. Zynq PS-ը պարամետրերը գրում է FPGA-ի AXI4-Lite ռեգիստրներում:
4. FPGA-ն սկսում է գեներացնել վիբրացիանման տվյալներ և կատարում իրական ժամանակի նախնական մշակում:
5. Տվյալները փոխանցվում են PS, ապա Ethernet-ով՝ սերվեր, և պահվում տվյալների բազայում:
6. Web dashboard-ը ցուցադրում է raw և processed ազդանշանները իրական ժամանակում:
7. FPGA-ն ներարկում է անոմալիա. FPGA-ի threshold detector-ը հայտնաբերում է իրադարձությունը, իսկ սերվերային մոդելը՝ նույնպես վերլուծում այն:
8. Հարթակը համեմատում է FPGA-ի հայտնաբերումը և ծրագրային վերլուծությունը:
9. Օգտվողը Web հարթակից փոխում է FPGA-ի պարամետրերը, և FPGA-ի վարքը փոխվում է **runtime**-ում՝ առանց bitstream վերագեներացնելու:

Վերջնական արդյունքը ուստի ոչ թե պարզ սենսորային dashboard է, այլ ամբողջական, կոնֆիգուրելի **Web Platform** $\leftrightarrow$ **Server** $\leftrightarrow$ **Zynq PS** $\leftrightarrow$ **FPGA** համակարգ, որն աջակցում է տվյալների գեներացում, հավաքագրում, իրական ժամանակի մշակում, կոնֆիգուրացիա, պահպանում, վիզուալիզացիա, վերլուծություն և ընտրովի ադապտիվ կառավարում:

Բաժին 11

Ընդլայնումներ (ընտրովի)

Ստորև բերվածները դիտվում են որպես **առաջադեմ ընդլայնումներ**, ոչ պարտադիր MVP պահանջներ.

- **AXI DMA / DDR ուղի** — բարձր տվյալահոսքի դեպքում AXI4-Stream → AXI DMA → DDR → PS:
- **FPGA reconfiguration** — Web հարթակից նախապես գեներացված կոնֆիգուրացիաների (bitstream-ների) միջև ընտրություն (օր.՝ Environmental / Vibration / High-Speed acquisition):
- **Ֆիզիկական սենսորներ** — էմուլացված աղբյուրների փոխարեն կամ դրանց կողքին:
- **Ադապտիվ հետադարձ կապ (closed loop)** — սերվերի մոդելը հայտնաբերում է վարքի փոփոխություն և ավտոմատ փոխում FPGA-ի պարամետրերը (threshold, filter window, sample rate, մշակման ռեժիմ)՝ ստեղծելով ադապտիվ ապարատա-ծրագրային համակարգ:
- **Linux Zynq PS-ի վրա** — սերվերային բաղադրիչների մասնակի տեղափոխում Zynq PS-ի Linux-ի վրա:

Ամփոփում

Առաջադրանքը սահմանում է միջամբիոնային դիպլոմային նախագիծ, որտեղ առաջին ուսանողը կենտրոնանում է FPGA/RTL ապարատի, իսկ երկրորդը՝ ծրագրային հարթակի ու տվյալների վերլուծության վրա, իսկ ընդհանուր համակարգը ապահովում է երկկողմանի հաղորդակցություն և runtime կոնֆիգուրացիա:

Հապավումների ցանկ

API Application Programming Interface.

AXI Advanced eXtensible Interface.

BRAM Block RAM.

DDR Double Data Rate memory.

DMA Direct Memory Access.

FIFO First-In First-Out.

FPGA Field-Programmable Gate Array.

IP Intellectual Property core.

LFSR Linear-Feedback Shift Register.

MPSoC Multiprocessor System-on-Chip.

MVP Minimum Viable Product.

PL Programmable Logic.

PS Processing System.

RTL Register-Transfer Level.

Բառարան

bitstream FPGA-ի կոնֆիգուրացիայի երկուական ֆայլ, որը ծրագրավորում է ծրագրավորվող տրամաբանությունը (PL).

experiment (փորձ) տվյալների հավաքագրման մեկ սեանս՝ որոշակի կոնֆիգուրացիայով, սկիզբ և ավարտ ունեցող.

runtime կոնֆիգուրացիա արդեն իրականացված ապարատի պարամետրերի փոփոխություն՝ առանց նոր bitstream գեներացնելու.